Local storage is best used for small, non-sensitive data that benefits from persistence across sessions and page reloads. Common examples include user preferences such as theme settings (like light or dark mode), font size, language selection, and UI layout preferences; application state data that needs to be restored when a user returns, such as form progress, selected filters, or UI state; cached data like API responses or content that can speed up subsequent visits or enable offline access; shopping cart contents for simple e-commerce sites, allowing items to be temporarily stored locally before checkout; and other non-sensitive settings such as notifications preferences, sidebar toggles, or other customizable options.

The best practice is to use local storage only for small, non-sensitive data—such as user preferences or application state—due to its typical storage limit of about 5–10MB per domain. The same-origin policy restricts access to data so that only pages from the same domain can read or modify it. Because all operations are synchronous, storing large amounts of data can block the main thread and degrade performance, so it should be avoided.

When `localStorage` storage is exceeded, browsers throw a `QuotaExceededError`. Use try/catch blocks to handle this gracefully, and upon error, remove outdated or less important data before retrying. Notify users if they approach or exceed limits and offer manual clearing options. Regular checks and proactive cleanup help avoid quota issues and maintain stability.

To manage data lifecycle in `localStorage`, including clearing outdated or unnecessary data, it is essential to implement strategies such as manual or scheduled deletion for specific entries or all data, as well as passive expiration by storing timestamps with each value and removing data when it exceeds a defined threshold. Granular control can be achieved by assigning expiration rules to groups of keys using prefixes or metadata, enabling tiered data retention policies. Additionally, triggering data cleanup during key application events—such as user logout or periods of inactivity—helps maintain storage efficiency, while providing user-controlled clearance options empowers individuals to manage their own data and privacy. Combining these approaches ensures that localStorage remains performant, relevant, and secure throughout its use.